



Advanced Data Structures NTUA 2007

R-trees and Grid File



Multi-dimensional Indexing

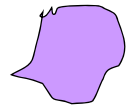
- GIS applications (maps):
 - Urban planning, route optimization, fire or pollution monitoring, utility networks, etc.
 - ESRI (ArcInfo), Oracle Spatial, etc.
- Other applications:
 - VLSI design, CAD/CAM, model of human brain, etc.
- Traditional applications:
 - Multidimensional records



Spatial data types


point


line


region

- Point : 2 real numbers
- Line : sequence of points
- Region : area included inside n-points



Spatial Relationships

- Topological relationships:
 - adjacent, inside, disjoint, etc
- Direction relationships:
 - Above, below, north_of, etc
- Metric relationships:
 - "distance < 100"
- And operations to express the relationships



Spatial Queries

- Selection queries: "Find all objects inside query q ", inside- $\rightarrow$ intersects, north
- Nearest Neighbor-queries: "Find the closets object to a query point q ", k -closest objects
- Spatial join queries: Two spatial relations $S1$ and $S2$, find all pairs: $\{x \text{ in } S1, y \text{ in } S2, \text{ and } x \text{ rel } y = \text{true}\}$, $\text{rel} = \text{intersect, inside, etc}$



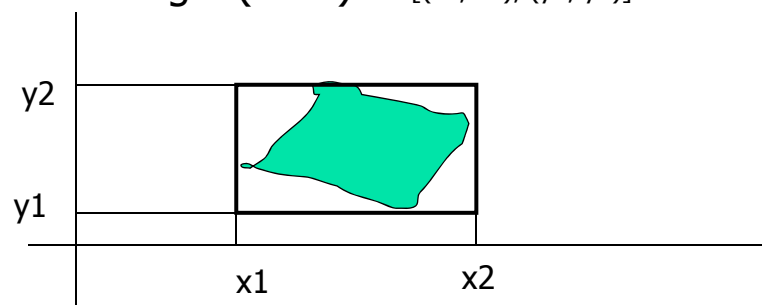
Access Methods

- Point Access Methods (PAMs):
 - Index methods for 2 or 3-dimensional points (k -d trees, Z-ordering, grid-file)
- Spatial Access Methods (SAMs):
 - Index methods for 2 or 3-dimensional regions and points (R-trees)



Indexing using SAMs

- Approximate each region with a simple shape: usually Minimum Bounding Rectangle (MBR) = $[(x1, x2), (y1, y2)]$



Indexing using SAMs (cont.)

Two steps:

- Filtering step: Find all the MBRs (using the SAM) that satisfy the query
- Refinement step: For each qualified MBR, check the original object against the query



Spatial Indexing

- Point Access Methods (PAMs) vs Spatial Access Methods (SAMs)
- PAM: index only point data
 - Hierarchical (tree-based) structures
 - Multidimensional Hashing
 - Space filling curve
- SAM: index both points and regions
 - Transformations
 - Overlapping regions
 - Clipping methods



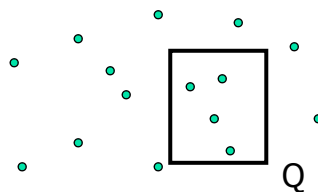
Spatial Indexing

Point Access Methods



The problem

- Given a point set and a rectangular query, find the points enclosed in the query
- We allow insertions/deletions on line

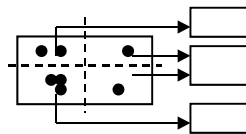
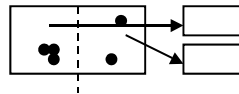
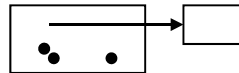


Grid File

- Hashing methods for multidimensional points (extension of Extensible hashing)
- Idea: Use a grid to partition the space → each cell is associated with one page
- Two disk access principle (exact match)

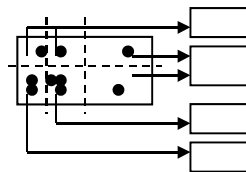
The Grid File: An Adaptable, Symmetric Multikey File Structure
J. NIEVERGELT, H. HINTERBERGER Institut für Informatik, ETH AND
K. C. SEVCIK University of Toronto. ACM TODS 1984.

Grid File



- Start with one bucket for the whole space.
- Select dividers along each dimension. Partition space into cells
- Dividers cut all the way.

Grid File

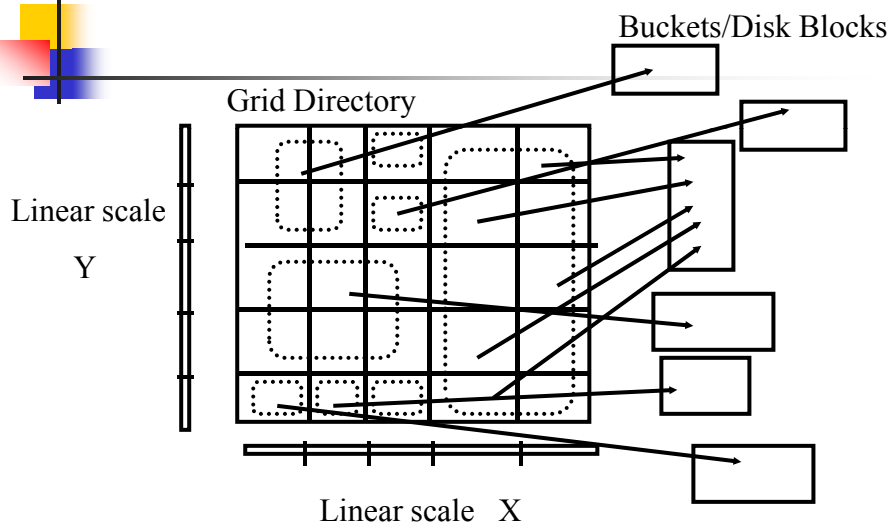


- Each cell corresponds to 1 disk page.
- Many cells can point to the same page.
- Cell directory potentially exponential in the number of dimensions

Grid File Implementation

- Dynamic structure using a grid directory
 - Grid array: a 2 dimensional array with pointers to buckets (this array can be large, disk resident) $G(0, \dots, nx-1, 0, \dots, ny-1)$
 - Linear scales: Two 1 dimensional arrays that used to access the grid array (main memory) $X(0, \dots, nx-1), Y(0, \dots, ny-1)$

Example





Grid File Search

- Exact Match Search: at most 2 I/Os assuming linear scales fit in memory.
 - First use linear scales to determine the index into the cell directory
 - access the cell directory to retrieve the bucket address (may cause 1 I/O if cell directory does not fit in memory)
 - access the appropriate bucket (1 I/O)
- Range Queries:
 - use linear scales to determine the index into the cell directory.
 - Access the cell directory to retrieve the bucket addresses of buckets to visit.
 - Access the buckets.



Grid File Insertions

- Determine the bucket into which insertion must occur.
- If space in bucket, insert.
- Else, split bucket
 - how to choose a good dimension to split?
 - ans: create convex regions for buckets.
- If bucket split causes a cell directory to split do so and adjust linear scales.
- insertion of these new entries potentially requires a complete reorganization of the cell directory---expensive!!!



Grid File Deletions

- Deletions may decrease the space utilization.
Merge buckets
- We need to decide which cells to merge and a merging threshold
- Buddy system and neighbor system
 - A bucket can merge with only one *buddy* in each dimension
 - Merge adjacent regions if the result is a rectangle

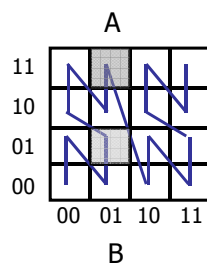


Z-ordering

- Basic assumption: Finite precision in the representation of each co-ordinate, K bits (2^K values)
- The address space is a square (image) and represented as a $2^K \times 2^K$ array
- Each element is called a pixel

Z-ordering

- Impose a linear ordering on the pixels of the image → 1 dimensional problem



$$Z_A = \text{shuffle}(x_A, y_A) = \text{shuffle}("01", "11")$$

$$= 0111 = (7)_{10}$$

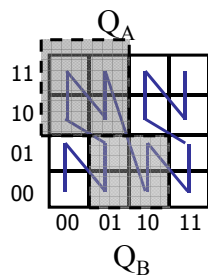
$$Z_B = \text{shuffle}("01", "01") = 0011$$

Z-ordering

- Given a point (x, y) and the precision K find the pixel for the point and then compute the z-value
- Given a set of points, use a B+-tree to index the z-values
- A range (rectangular) query in 2-d is mapped to a set of ranges in 1-d

Queries

- Find the z-values that contained in the query and then the ranges



$Q_A \rightarrow \text{range } [4, 7]$

$Q_B \rightarrow \text{ranges } [2,3] \text{ and } [8,9]$

Hilbert Curve

- We want points that are close in 2d to be close in the 1d
- Note that in 2d there are 4 neighbors for each point where in 1d only 2.
- Z-curve has some "jumps" that we would like to avoid
- Hilbert curve avoids the jumps : recursive definition

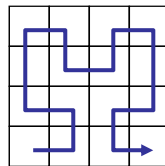


Hilbert Curve- example

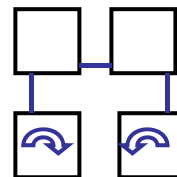
- It has been shown that in general Hilbert is better than the other space filling curves for retrieval [Jag90]
- H_i (order- i) Hilbert curve for $2^i \times 2^i$ array



H1



H2



... H(n+1)



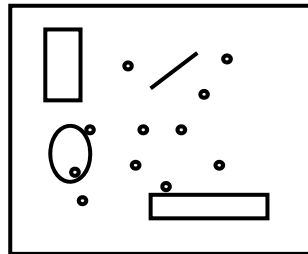
Reference

- H. V. Jagadish: Linear Clustering of Objects with Multiple Attributes. ACM SIGMOD Conference 1990: 332-342



Problem

- Given a collection of geometric objects (points, lines, polygons, ...)
- organize them on disk, to answer spatial queries (range, nn, etc)



R-trees

- [Guttman 84] Main idea: extend B+-tree to multi-dimensional spaces!
- (only deal with Minimum Bounding Rectangles - **MBRs**)

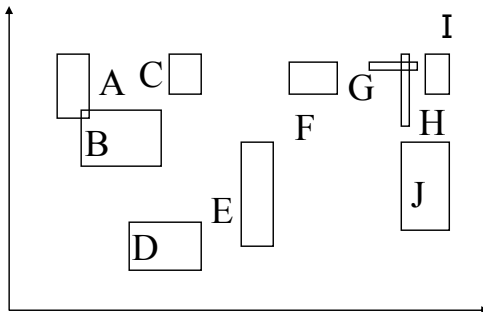


R-trees

- A multi-way external memory tree
- Index nodes and data (leaf) nodes
- All leaf nodes appear on the same level
- Every node contains between t and M entries
- The root node has at least 2 entries (children)

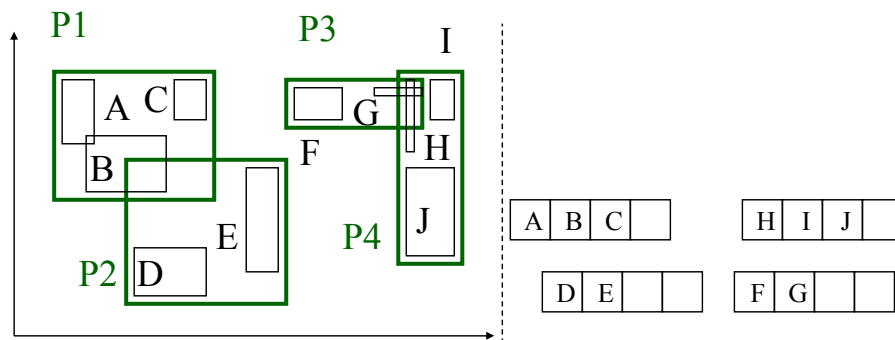
Example

- eg., w/ fanout 4: group nearby rectangles to parent MBRs; each group $\rightarrow$ disk page



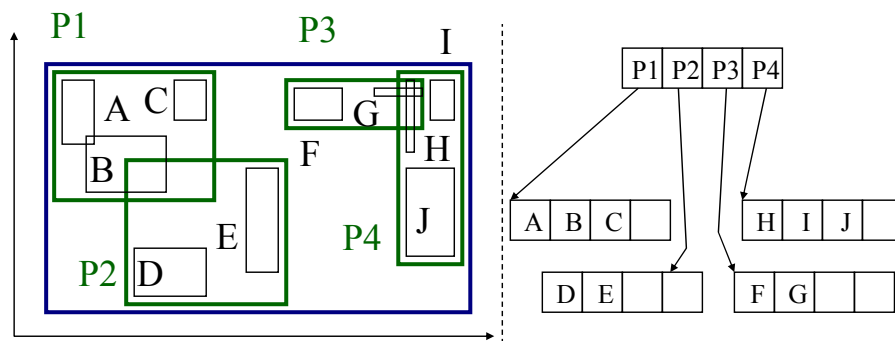
Example

■ F=4



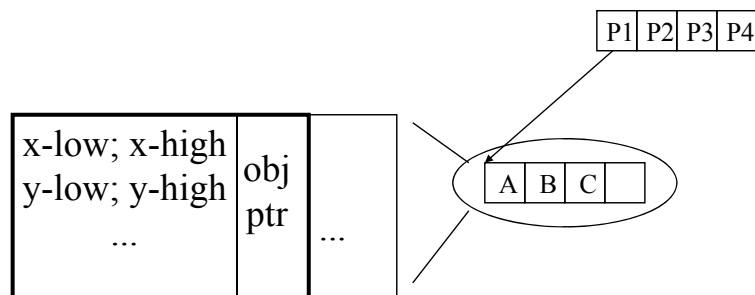
Example

■ F=4



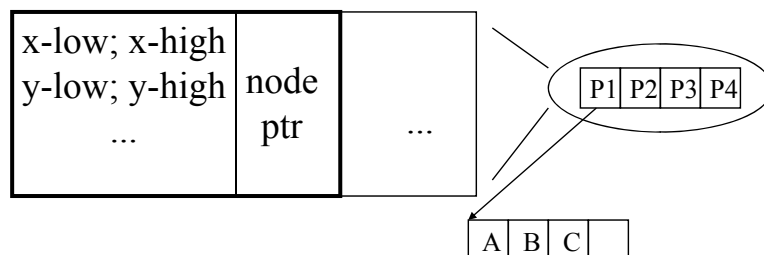
R-trees - format of nodes

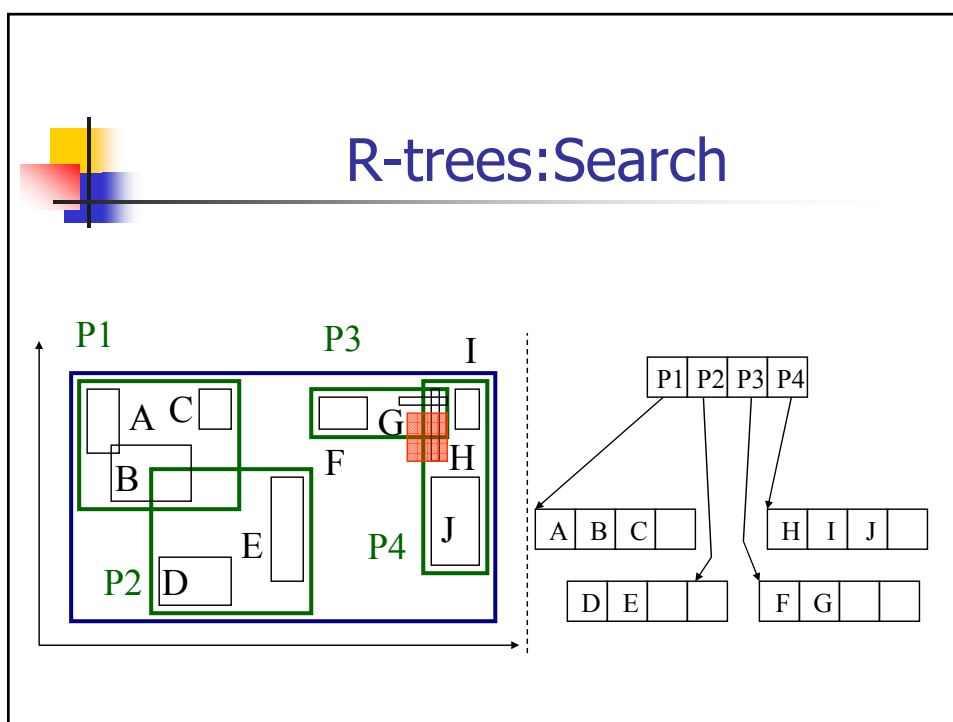
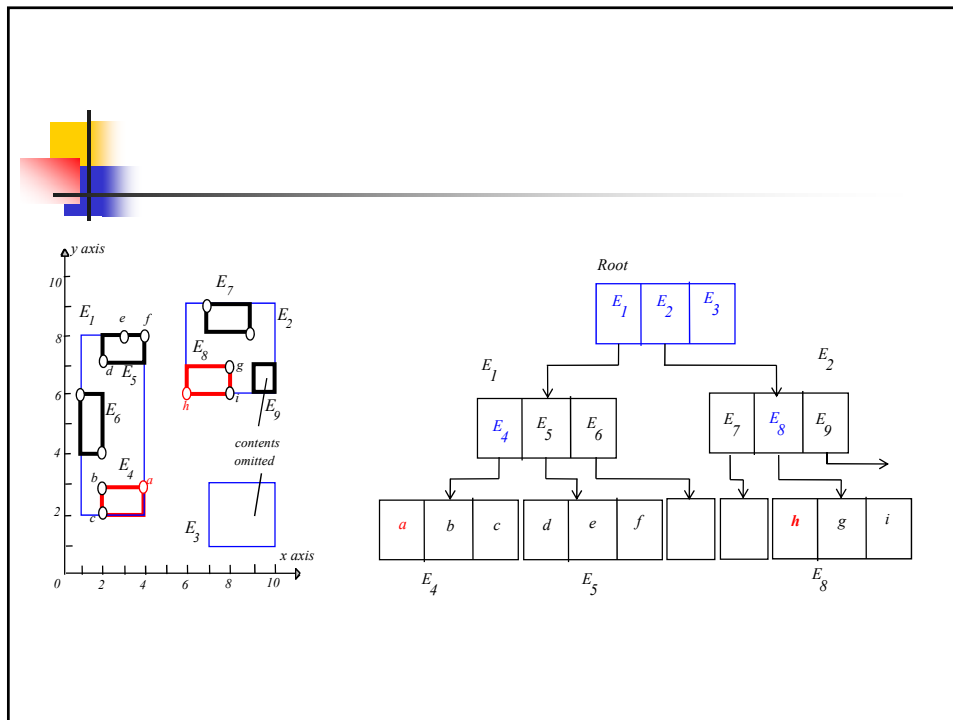
- $\{(MBR; \text{obj_ptr})\}$ for leaf nodes



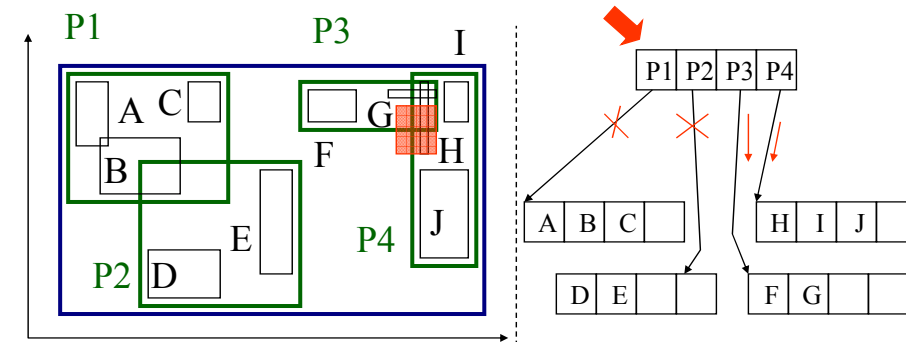
R-trees - format of nodes

- $\{(MBR; \text{node_ptr})\}$ for non-leaf nodes





R-trees: Search



R-trees: Search

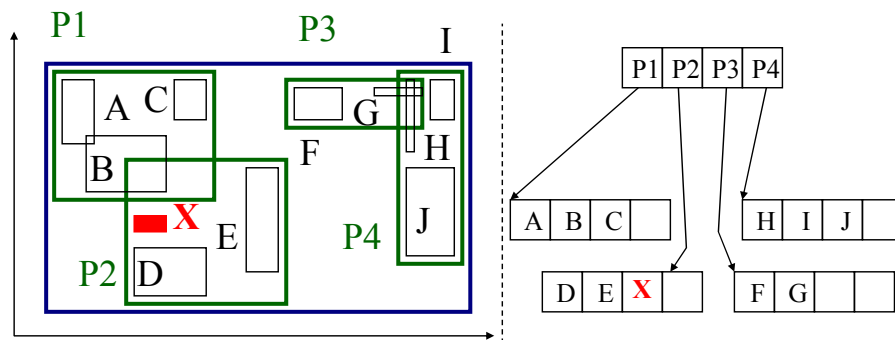
■ Main points:

- every parent node completely covers its 'children'
- a child MBR may be covered by more than one parent - it is stored under ONLY ONE of them. (ie., no need for dup. elim.)
- a point query may follow multiple branches.
- everything works for **any(?)** dimensionality



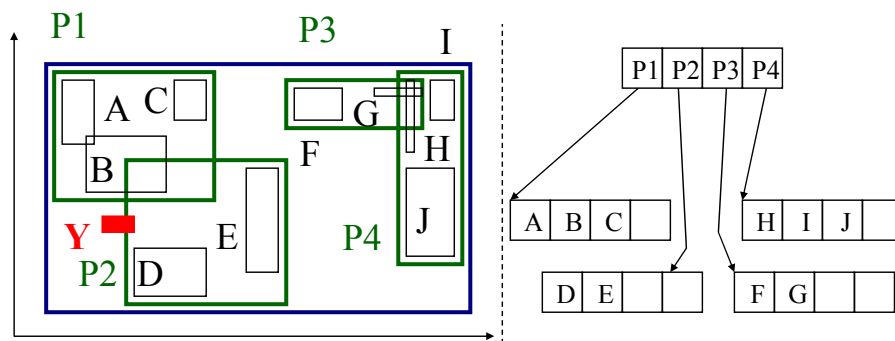
R-trees: Insertion

Insert X



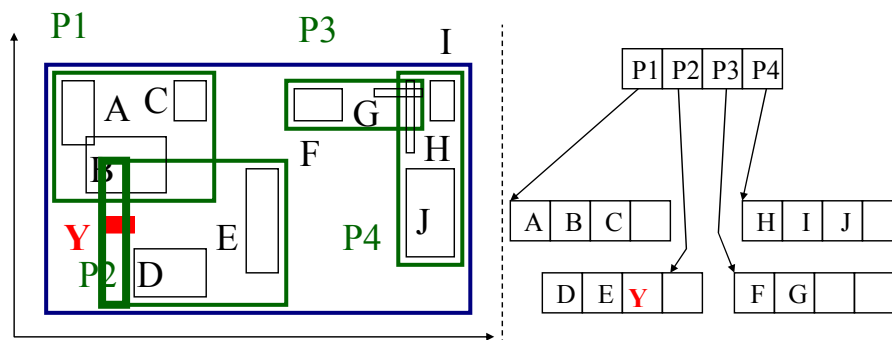
R-trees: Insertion

Insert Y



R-trees: Insertion

- Extend the parent MBR

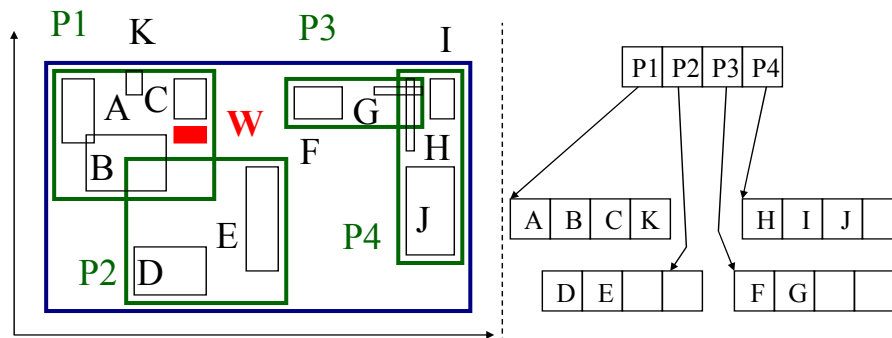


R-trees: Insertion

- How to find the next node to insert the new object?
 - Using ChooseLeaf: Find the entry that needs the least enlargement to include Y. Resolve ties using the area (smallest)
- Other methods (later)

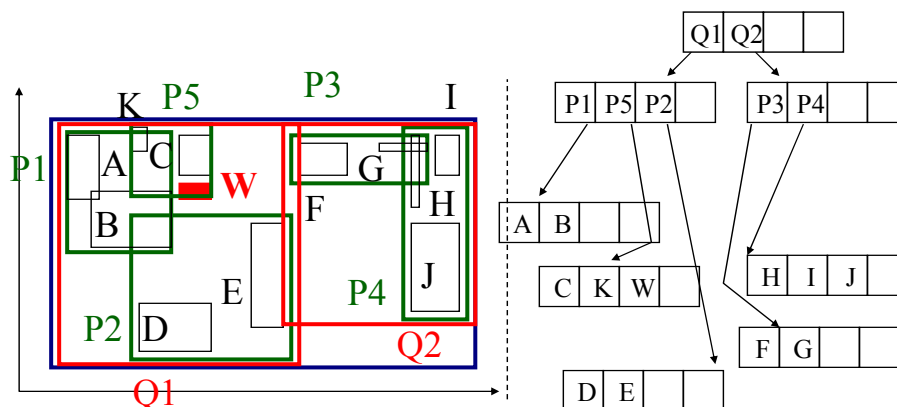
R-trees: Insertion

- If node is full then Split : ex. Insert w



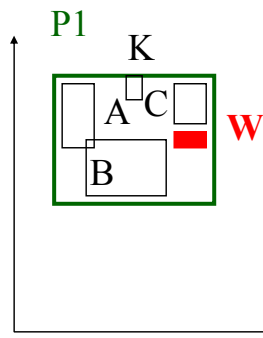
R-trees: Insertion

- If node is full then Split : ex. Insert w



R-trees:Split

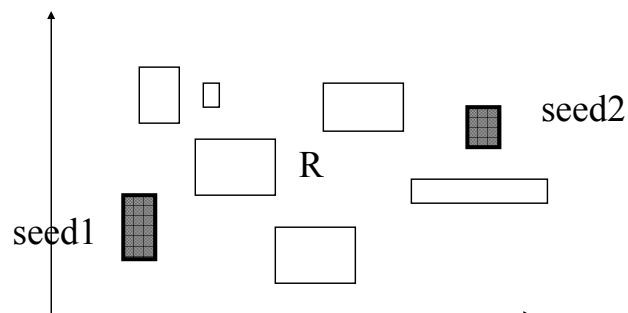
- Split node P1: partition the MBRs into two groups.



- (A1: plane sweep, until 50% of rectangles)
- A2: 'linear' split
- A3: quadratic split
- A4: exponential split:
 2^{M-1} choices

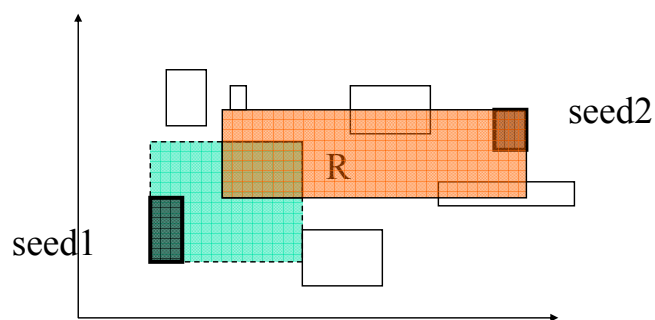
R-trees:Split

- pick two rectangles as 'seeds';
- assign each rectangle 'R' to the 'closest' 'seed'



R-trees: Split

- pick two rectangles as 'seeds';
- assign each rectangle 'R' to the 'closest' 'seed':
- 'closest': the smallest increase in area



R-trees: Split

- How to pick Seeds:
 - Linear: Find the highest and lowest side in each dimension, normalize the separations, choose the pair with the greatest normalized separation
 - Quadratic: For each pair E1 and E2, calculate the rectangle $J = \text{MBR}(E1, E2)$ and $d = J - E1 - E2$. Choose the pair with the largest d



R-trees: Insertion

- Use the **ChooseLeaf** to find the leaf node to insert an entry E
- If leaf node is full, then **Split**, otherwise insert there
 - Propagate the split upwards, if necessary
- Adjust parent nodes



R-Trees: Deletion

- Find the leaf node that contains the entry E
- Remove E from this node
- If underflow:
 - Eliminate the node by removing the node entries and the parent entry
 - Reinsert the orphaned (other entries) into the tree using **Insert**
- Other method (**later**)



R-trees: Variations

- R+-tree: DO not allow overlapping, so split the objects (similar to z-values)
Greek R-tree (Faloutsos, Roussopoulos, Sellis)
- R*-tree: change the insertion, deletion algorithms (minimize not only area but also perimeter, forced re-insertion)
German R-tree: Kriegel's group
- Hilbert R-tree: use the Hilbert values to insert objects into the tree



R-tree

- The original R-tree tries to minimize the area of each enclosing rectangle in the index nodes.
- Is there any other property that can be optimized?

R*-tree → Yes!



R*-tree

- Optimization Criteria:
 - (O1) Area covered by an index MBR
 - (O2) Overlap between index MBRs
 - (O3) Margin of an index rectangle
 - (O4) Storage utilization
- Sometimes it is impossible to optimize all the above criteria at the same time!



R*-tree

- ChooseSubtree:
 - If next node is a leaf node, choose the node using the following criteria:
 - Least overlap enlargement
 - Least area enlargement
 - Smaller area
 - Else
 - Least area enlargement
 - Smaller area

R*-tree



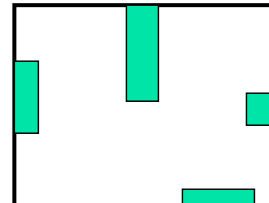
SplitNode

- Choose the axis to split
 - Choose the two groups along the chosen axis
- ChooseSplitAxis
 - Along each axis, sort rectangles and break them into two groups ($M-2m+2$ possible ways where one group contains at least m rectangles). Compute the sum S of all margin-values (perimeters) of each pair of groups. Choose the one that minimizes S
- ChooseSplitIndex
 - Along the chosen axis, choose the grouping that gives the minimum overlap-value

R*-tree



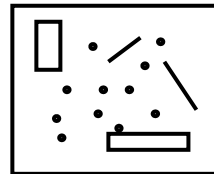
- Forced Reinsert:
 - defer splits, by forced-reinsert, i.e.: instead of splitting, temporarily delete some entries, shrink overflowing MBR, and re-insert those entries
- Which ones to re-insert?
- How many? A: 30%





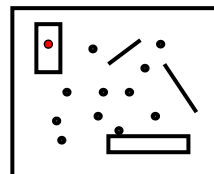
Spatial Queries

- Given a collection of geometric objects (points, lines, polygons, ...)
- organize them on disk, to answer efficiently
 - point queries
 - range queries
 - k-nn queries
 - spatial joins ('all pairs' queries)



Spatial Queries

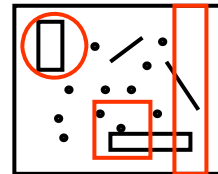
- Given a collection of geometric objects (points, lines, polygons, ...)
- organize them on disk, to answer
 - point queries
 - range queries
 - k-nn queries
 - spatial joins ('all pairs' queries)





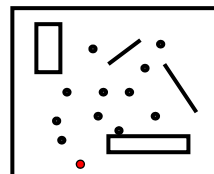
Spatial Queries

- Given a collection of geometric objects (points, lines, polygons, ...)
- organize them on disk, to answer
 - point queries
 - range queries
 - k-nn queries
 - spatial joins ('all pairs' queries)



Spatial Queries

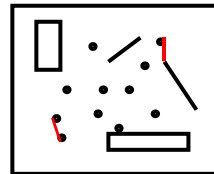
- Given a collection of geometric objects (points, lines, polygons, ...)
- organize them on disk, to answer
 - point queries
 - range queries
 - k-nn queries
 - spatial joins ('all pairs' queries)



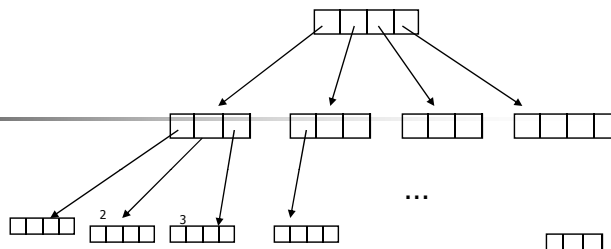


Spatial Queries

- Given a collection of geometric objects (points, lines, polygons, ...)
- organize them on disk, to answer
 - point queries
 - range queries
 - k-nn queries
 - spatial joins ('all pairs' queries)



R-tree



R-trees - Range search

pseudocode:

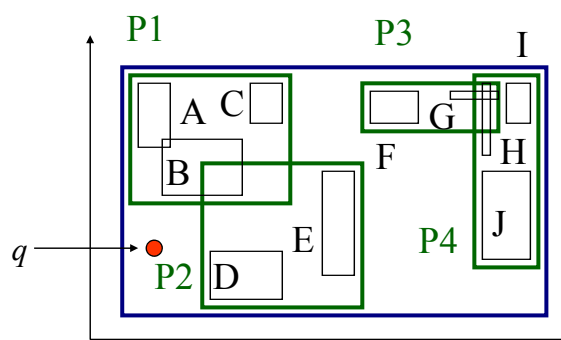
check the root

for each branch,

if its MBR intersects the query rectangle

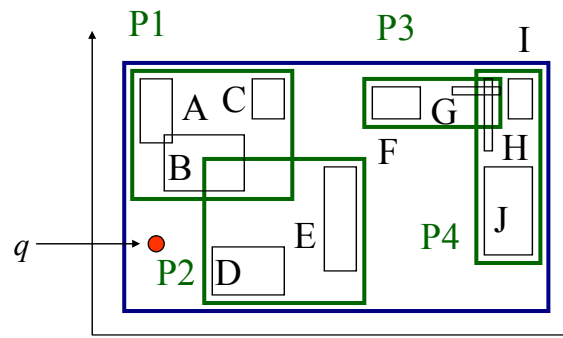
apply range-search (or print out, if this
is a leaf)

R-trees - NN search



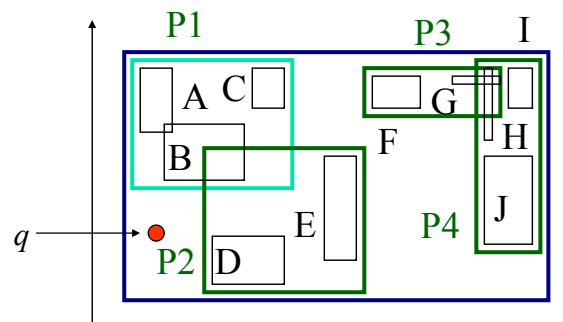
R-trees - NN search

- Q: How? (find near neighbor; refine...)



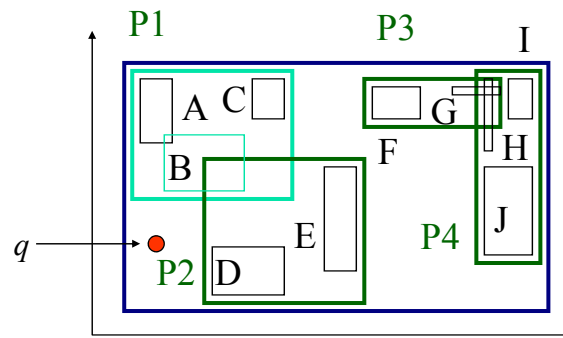
R-trees - NN search

- A1: depth-first search; then range query



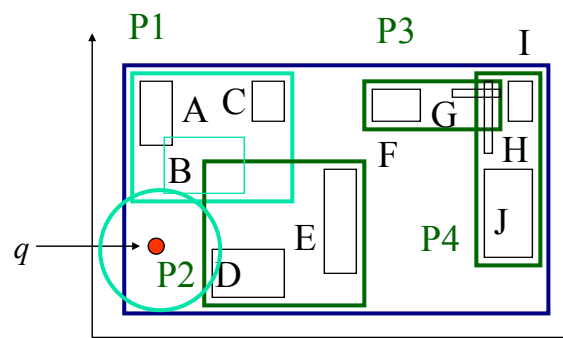
R-trees - NN search

- A1: depth-first search; then range query



R-trees - NN search

- A1: depth-first search; then range query





R-trees - NN search: Branch and Bound

- A2: [Roussopoulos+, sigmod95]:
 - At each node, priority queue, with promising MBRs, and their best and worst-case distance
- main idea: Every face of any MBR contains at least one point of an actual spatial object!



MBR face property

- MBR is a d-dimensional rectangle, which is the minimal rectangle that fully encloses (bounds) an object (or a set of objects)
- MBR f.p.: Every face of the MBR contains at least one point of some object in the database



Search improvement

- Visit an MBR (node) only when necessary
- How to do pruning? Using MINDIST and MINMAXDIST

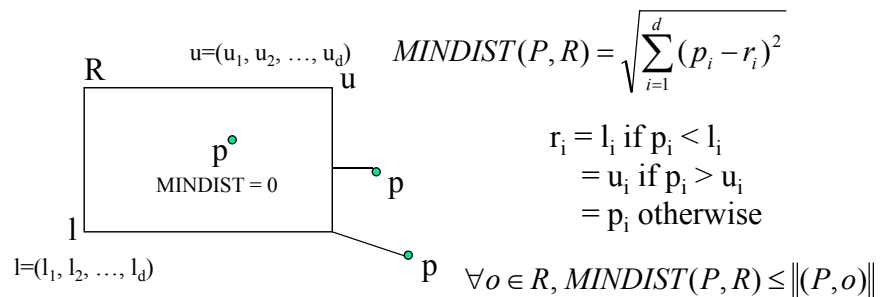


MINDIST

- $MINDIST(P, R)$ is the minimum distance between a point P and a rectangle R
- If the point is inside R , then $MINDIST=0$
- If P is outside of R , $MINDIST$ is the distance of P to the closest point of R (one point of the perimeter)

MINDIST computation

- $MINDIST(p, R)$ is the minimum distance between p and R with corner points l and u
 - the closest point in R is at least this distance away



R $u=(u_1, u_2, \dots, u_d)$ $MINDIST(P, R) = \sqrt{\sum_{i=1}^d (p_i - r_i)^2}$
 $l=(l_1, l_2, \dots, l_d)$ l u p p
 $MINDIST = 0$ $r_i = l_i$ if $p_i < l_i$
 $= u_i$ if $p_i > u_i$
 $= p_i$ otherwise
 $\forall o \in R, MINDIST(P, R) \leq \|(P, o)\|$

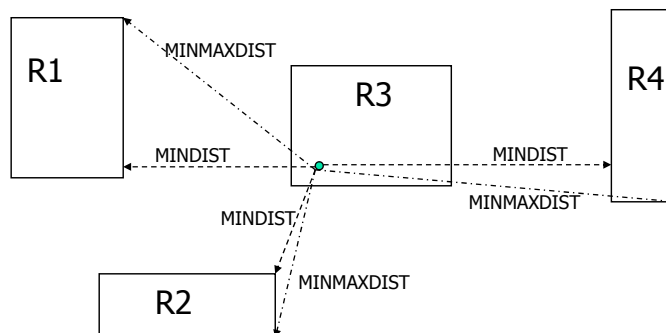
MINMAXDIST

- $MINMAXDIST(P, R)$: for each dimension, find the closest face, compute the distance to the furthest point on this face and take the minimum of all these (d) distances
- $MINMAXDIST(P, R)$ is the smallest possible upper bound of distances from P to R
- $MINMAXDIST$ guarantees that there is at least one object in R with a distance to P smaller or equal to it.

$$\exists o \in R, \|(P, o)\| \leq MINMAXDIST(P, R)$$

MINDIST and MINMAXDIST

- $\text{MINDIST}(P, R) \leq \text{NN}(P) \leq \text{MINMAXDIST}(P, R)$

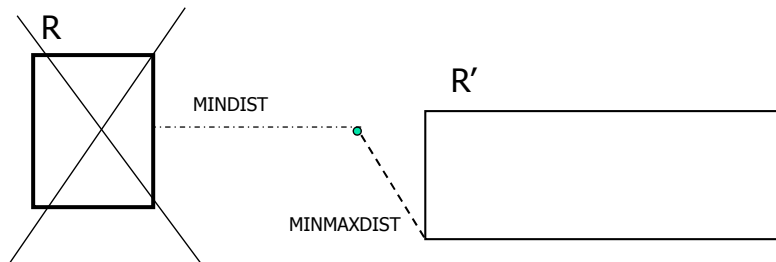


Pruning in NN search

- Downward pruning: An MBR R is discarded if there exists another R' s.t. $\text{MINDIST}(P, R) > \text{MINMAXDIST}(P, R')$
- Downward pruning: An object O is discarded if there exists an R s.t. the $\text{Actual-Dist}(P, O) > \text{MINMAXDIST}(P, R)$
- Upward pruning: An MBR R is discarded if an object O is found s.t. the $\text{MINDIST}(P, R) > \text{Actual-Dist}(P, O)$

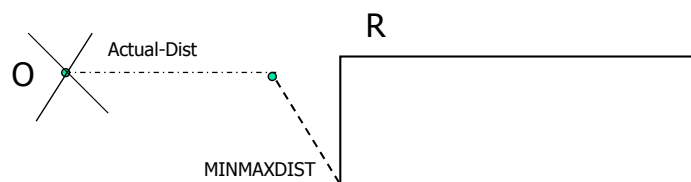
Pruning 1 example

- Downward pruning: An MBR R is discarded if there exists another R' s.t. $\text{MINDIST}(P, R) > \text{MINMAXDIST}(P, R')$



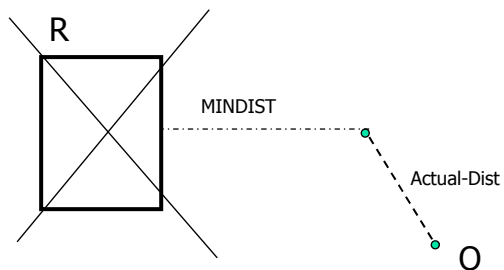
Pruning 2 example

- Downward pruning: An object O is discarded if there exists an R s.t. the $\text{Actual-Dist}(P, O) > \text{MINMAXDIST}(P, R)$



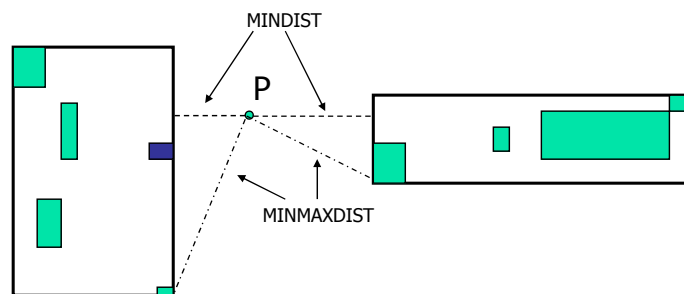
Pruning 3 example

- Upward pruning: An MBR R is discarded if an object O is found s.t. the $MINDIST(P,R) > Actual-Dist(P,O)$



Ordering Distance

- MINDIST is an optimistic distance where MINMAXDIST is a pessimistic one.





NN-search Algorithm

1. Initialize the nearest distance as infinite distance
2. Traverse the tree depth-first starting from the root. At each Index node, sort all MBRs using an ordering metric and put them in an **Active Branch List (ABL)**.
3. Apply pruning rules 1 and 2 to ABL
4. Visit the MBRs from the ABL following the order until it is empty
5. If Leaf node, compute actual distances, compare with the best NN so far, update if necessary.
6. At the return from the recursion, use pruning rule 3
7. When the ABL is empty, the NN search returns.



K-NN search

- Keep the sorted buffer of at most k current nearest neighbors
- Pruning is done using the k-th distance



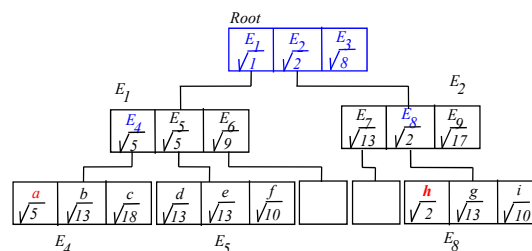
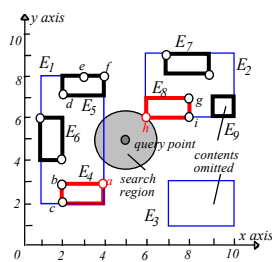
Another NN search: Best-First

- Global order [HS99]
 - Maintain distance to all entries in a common Priority Queue
 - Use only MINDIST
 - Repeat
 - Inspect the next MBR in the list
 - Add the children to the list and reorder
 - Until all remaining MBRs can be pruned



Nearest Neighbor Search (NN) with R-Trees

■ *Best-first* (BF) algorithm:



Action	Heap	Result
Visit Root	$E_1 \sqrt{1}$ $E_2 \sqrt{2}$ $E_3 \sqrt{8}$	{empty}
follow E_1	$E_4 \sqrt{5}$ $E_5 \sqrt{5}$ $E_6 \sqrt{9}$	{empty}
follow E_2	$E_7 \sqrt{13}$ $E_8 \sqrt{2}$ $E_9 \sqrt{17}$	{empty}
follow E_8	$E_4 \sqrt{5}$ $E_5 \sqrt{5}$ $E_6 \sqrt{9}$ $E_7 \sqrt{13}$ $E_9 \sqrt{17}$	{(h, $\sqrt{2}$)}
	$E_4 \sqrt{5}$ $E_5 \sqrt{5}$ $E_6 \sqrt{9}$ $E_7 \sqrt{13}$ $E_9 \sqrt{17}$	

Report h and terminate



HS algorithm

```
Initialize PQ (priority queue)
InsertQueue(PQ, Root)
While not IsEmpty(PQ)
    R = Dequeue(PQ)
    If R is an object
        Report R and exit (done!)
    If R is a leaf page node
        For each O in R, compute the Actual-Dists, InsertQueue(PQ, O)
    If R is an index node
        For each MBR C, compute MINDIST, insert into PQ
```



Best-First vs Branch and Bound

- Best-First is the “optimal” algorithm in the sense that it visits all the necessary nodes and nothing more!
- But needs to store a large Priority Queue in main memory. If PQ becomes large, we have thrashing...
- BB uses small Lists for each node. Also uses MINMAXDIST to prune some entries



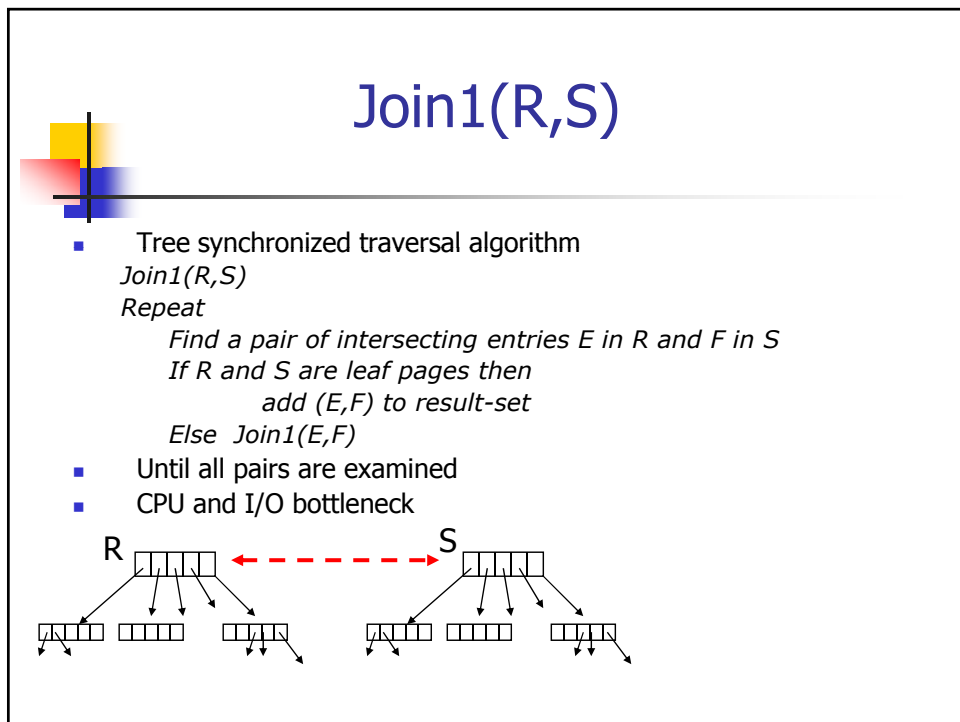
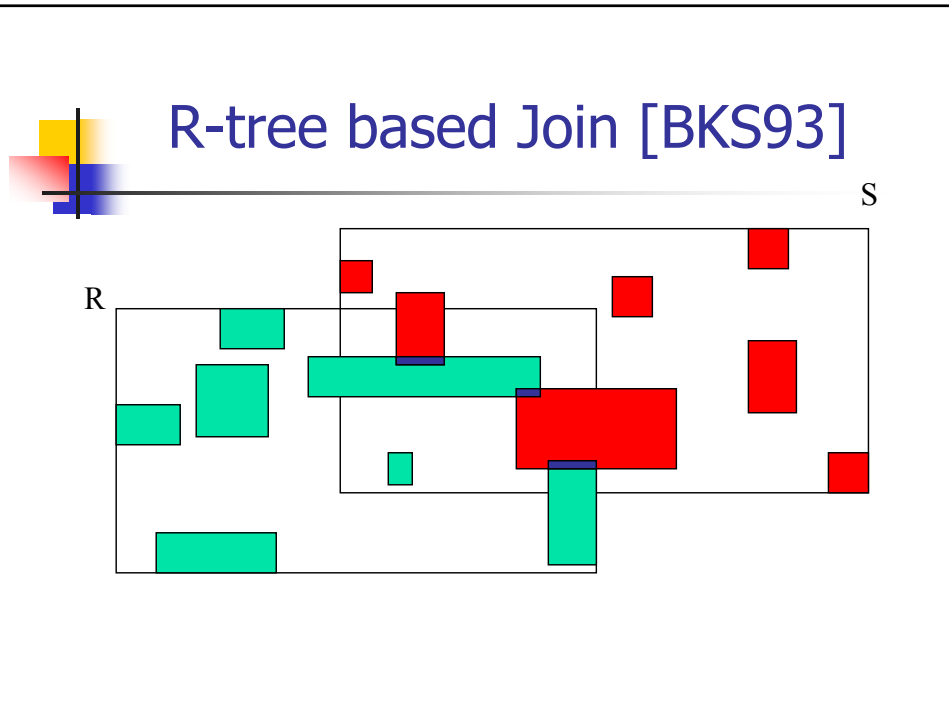
Spatial Join

- Find all parks in each city in MA
- Find all trails that go through a forest in MA
- Basic operation
 - find all pairs of objects that overlap
- Single-scan queries
 - nearest neighbor queries, range queries
- Multiple-scan queries
 - spatial join



Algorithms

- No existing index structures
 - Transform data into 1-d space [O89]
 - z-transform; sensitive to size of pixel
 - Partition-based spatial-merge join [PW96]
 - partition into tiles that can fit into memory
 - plane sweep algorithm on tiles
 - Spatial hash joins [LR96, KS97]
 - Sort data using recursive partitioning [BBKK01]
- With index structures [BKS93, HJR97]
 - k-d trees and grid files
 - R-trees



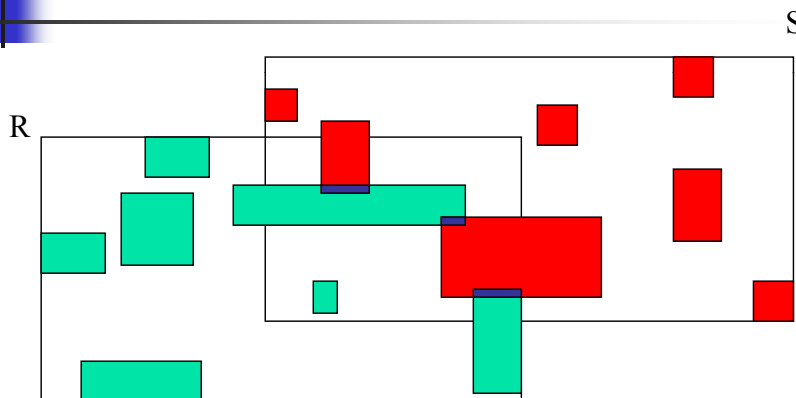


CPU – Time Tuning

- Two ways to improve CPU – time
 - Restricting the search space
 - Spatial sorting and plane sweep



Reducing CPU bottleneck



Join2(R,S,IntersectedVol)

Join2(R,S,IV)

Repeat

Find a pair of intersecting entries E in R and F in S that overlap with IV

*If R and S are leaf pages then
add (E,F) to result-set*

Else Join2(E,F,CommonEF)

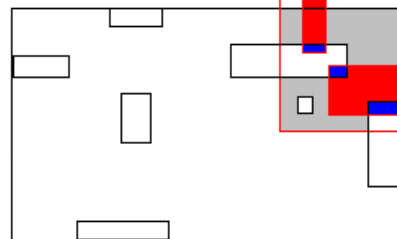
- Until all pairs are examined
- In general, number of comparisons equals
 - $\text{size}(R) + \text{size}(S) + \text{relevant}(R) * \text{relevant}(S)$
- Reduce the product term

Restricting the search space

Join1: 7 of R * 7 of S

= 49 comparisons

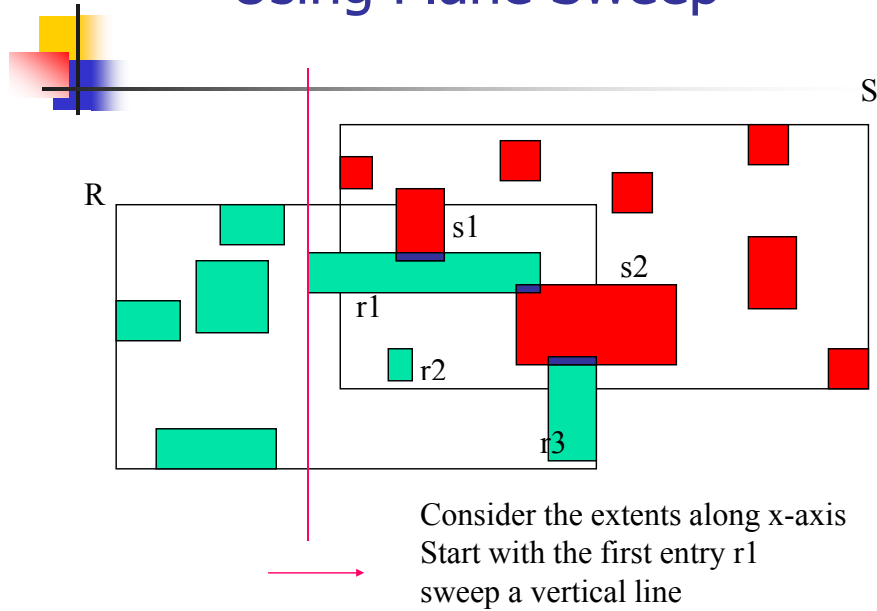
R



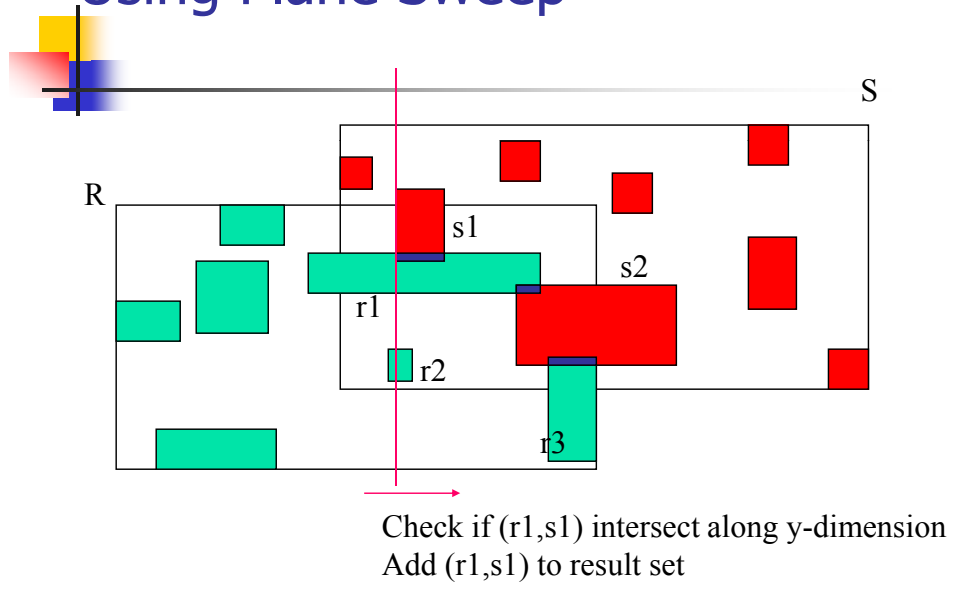
Now: 3 of R * 2 of S
= 6 comp

Plus Scanning:
7 of R + 7 of S
= 14 comp

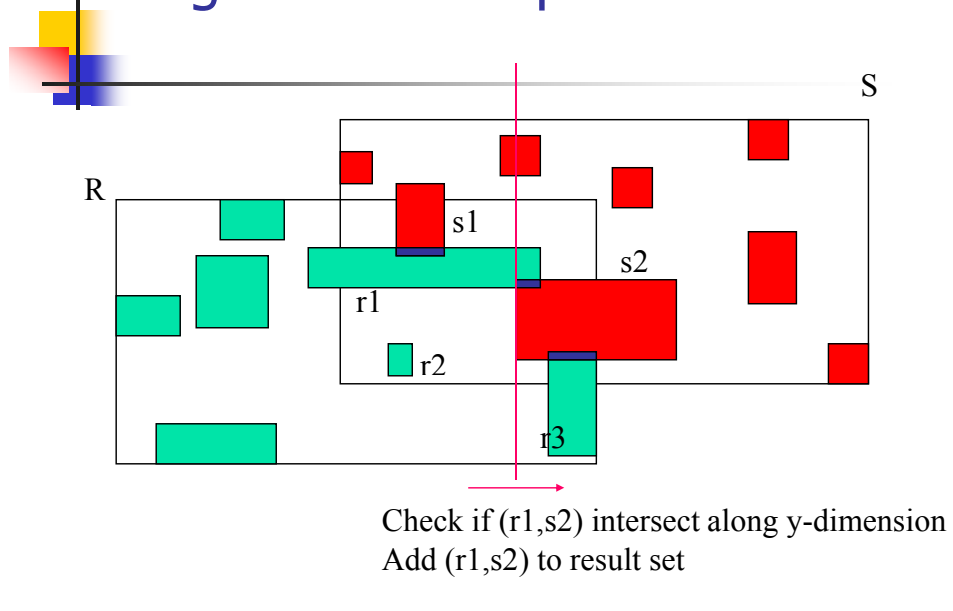
Using Plane Sweep



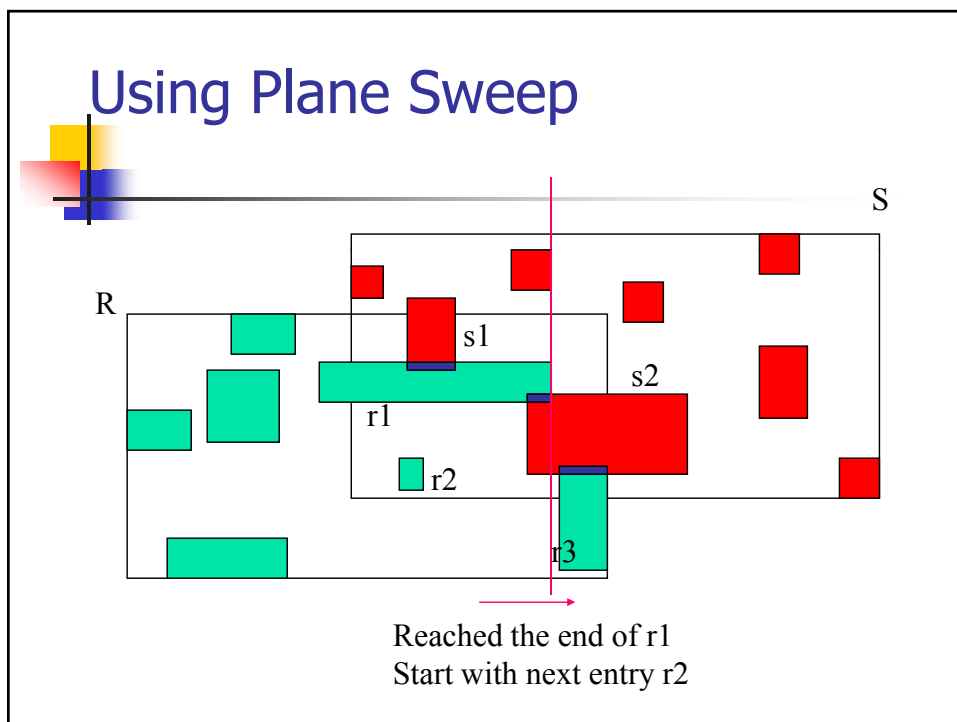
Using Plane Sweep



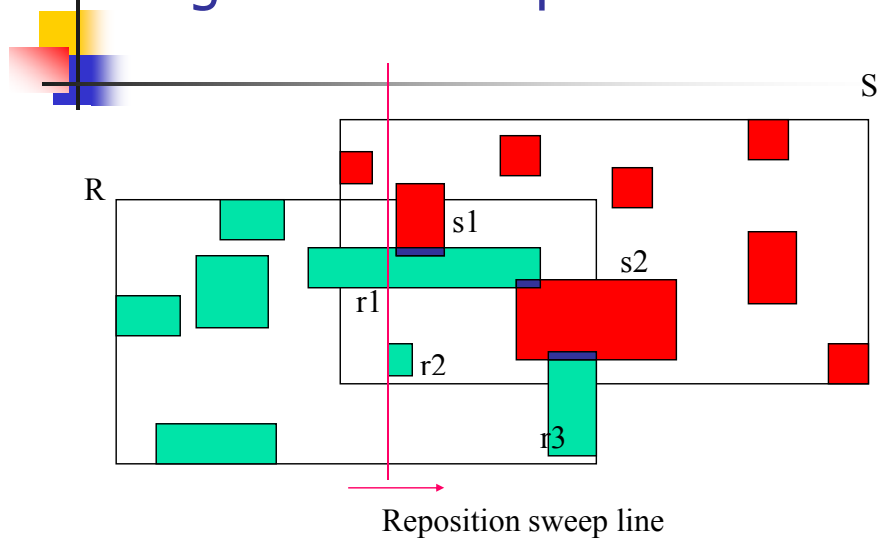
Using Plane Sweep



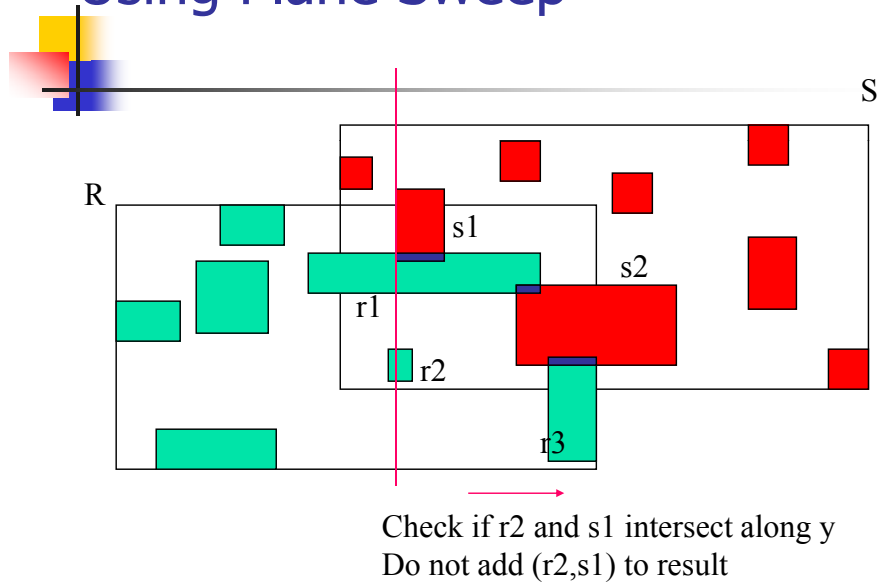
Using Plane Sweep



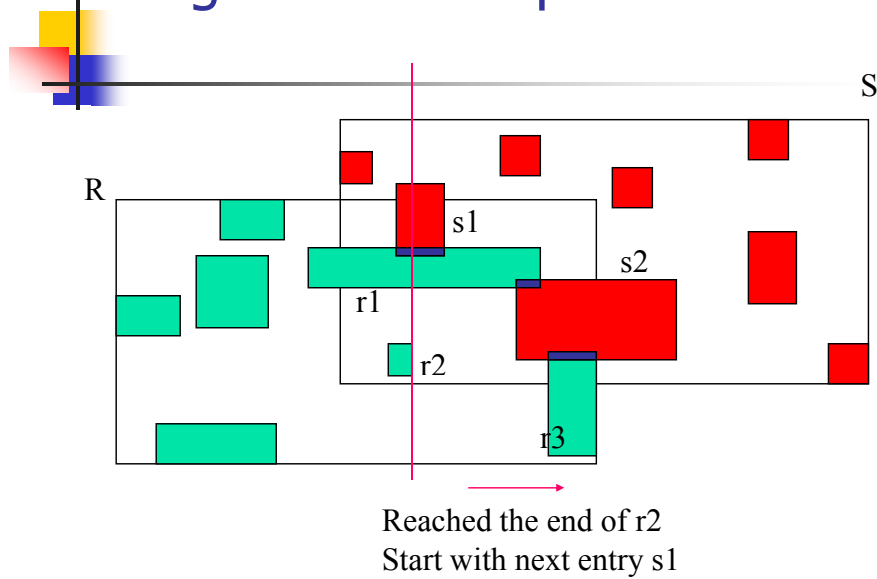
Using Plane Sweep



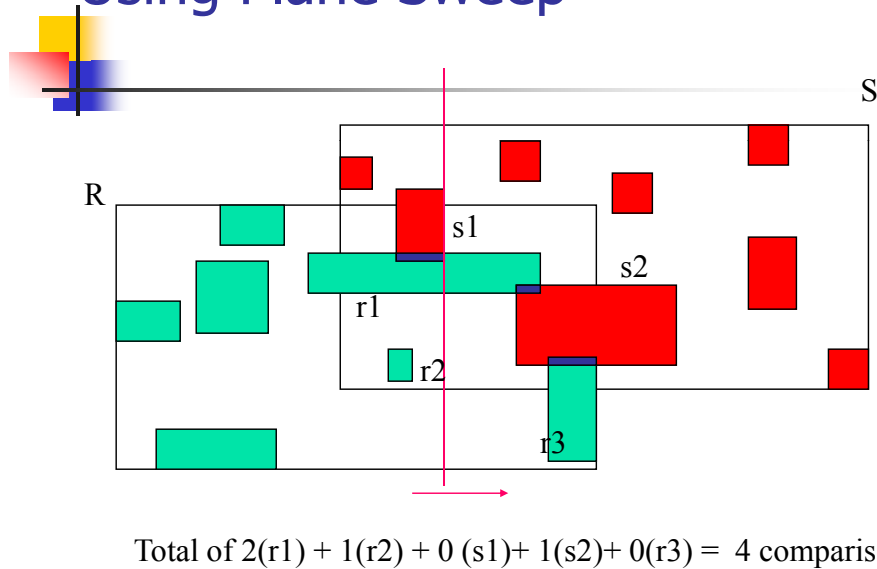
Using Plane Sweep



Using Plane Sweep



Using Plane Sweep





I/O Tunning

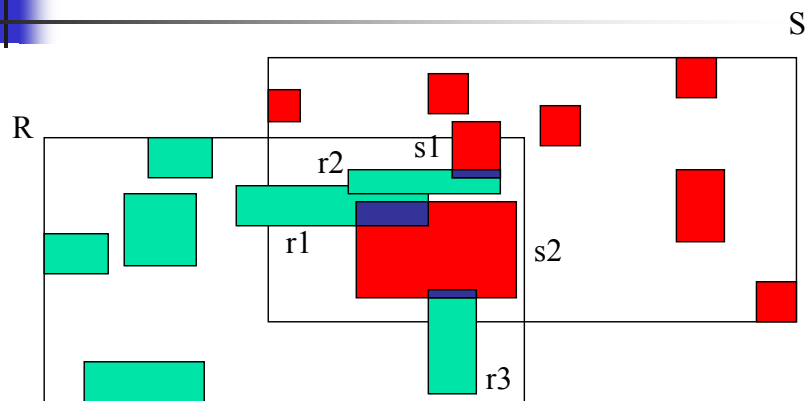
- Compute a read schedule of the pages to minimize the number of disk accesses
 - Local optimization policy based on spatial locality
- Three methods
 - Local plane sweep
 - Local plane sweep with pinning
 - Local z-order



Reducing I/O

- Plane sweep again:
 - Read schedule r_1, s_1, s_2, r_3
 - Every subtree examined only once
 - Consider a slightly different layout

Reducing I/O

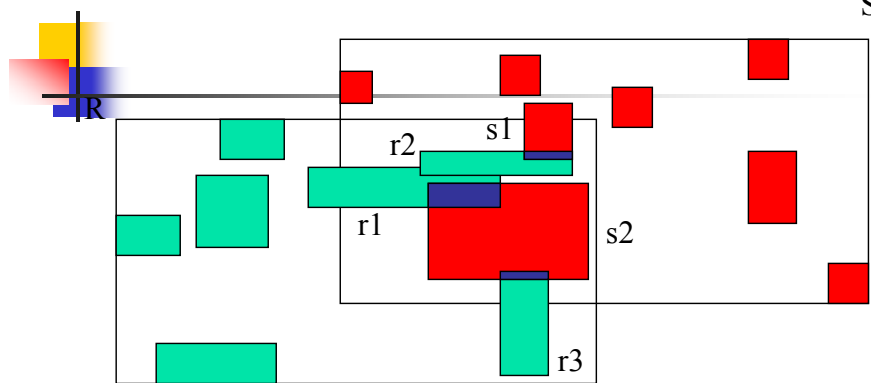


Read schedule is r1, s2, r2, s1, s2, r3
Subtree s2 is examined twice

Pinning of nodes

- After examining a pair (E,F), compute the degree of intersection of each entry
 - degree(E) is the number of intersections between E and **unprocessed** rectangles of the other dataset
- If the degrees are non-zero, pin the pages of the entry with maximum degree
- Perform spatial joins for this page
- Continue with plane sweep

Reducing I/O



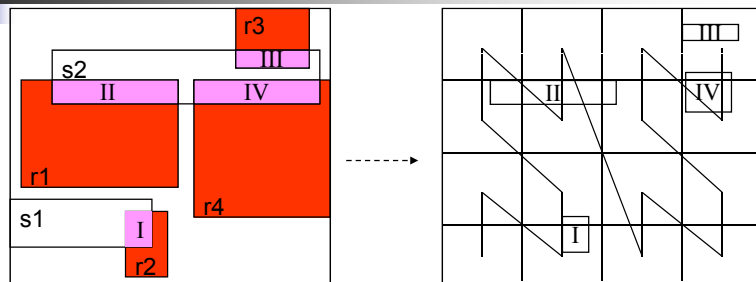
After computing $\text{join}(r1, s2)$,
 $\text{degree}(r1) = 0$
 $\text{degree}(s2) = 1$
 So, examine $s2$ next
 Read schedule = $r1, s2, r3, r2, s1$
 Subtree $s2$ examined only once

Local Z-Order

■ Idea:

1. Compute the intersections between each rectangle of the one node and all rectangles of the other node
2. Sort the rectangles according to the Z-ordering of their centers
3. Use this ordering to fetch pages

Local Z-ordering



Read schedule:
<s1,r2,r1,s2,r4,r3>

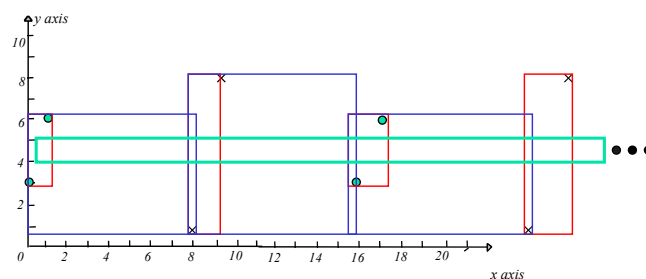
R-trees - performance analysis

- How many disk (=node) accesses we'll need for
 - range
 - nn
 - spatial joins
- Worst Case vs. Average Case

Worst Case Performance

- In the worst case, we need to perform $O(N/B)$ I/O's for an empty query (pretty bad!)
- We need to show a family of datasets and queries where any R-tree will perform like that

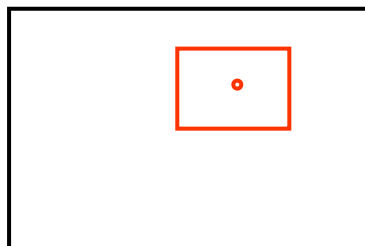
Example:





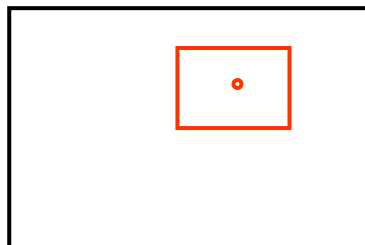
Average Case analysis

- How many disk accesses (expected value) for range queries?
 - query distribution wrt location?
 - " " wrt size?



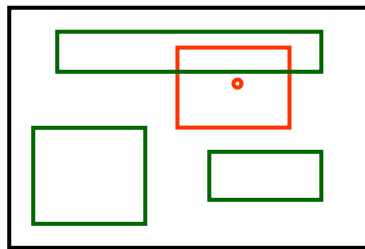
R-trees - performance analysis

- How many disk accesses for range queries?
 - query distribution wrt location? **uniform; (biased)**
 - " " wrt size? **uniform**



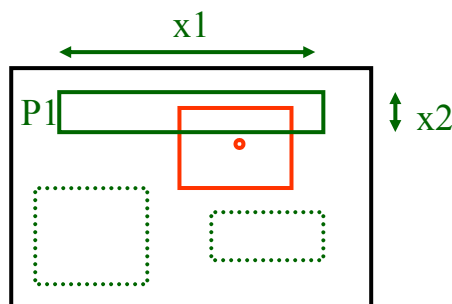
R-trees - performance analysis

- easier case: we know the positions of data nodes and their MBRs, eg:



R-trees - performance analysis

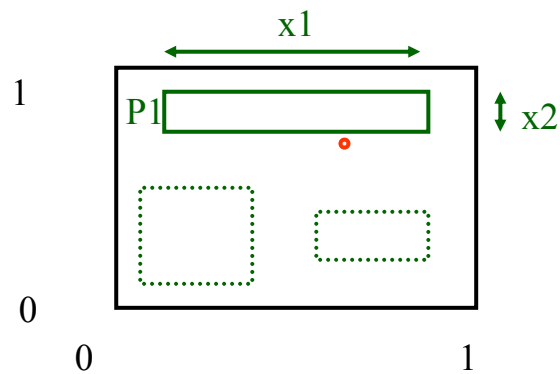
- How many times will P1 be retrieved (unif. queries)?





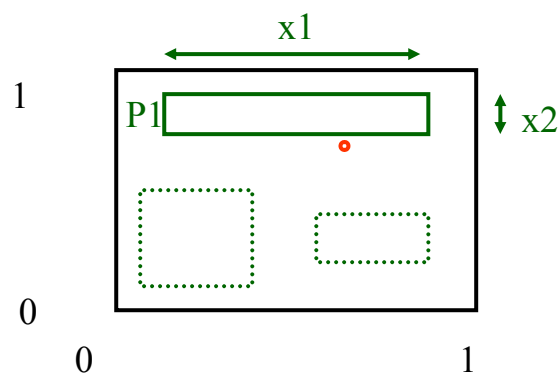
R-trees - performance analysis

How many times will P1 be retrieved (unif. POINT queries)?



R-trees - performance analysis

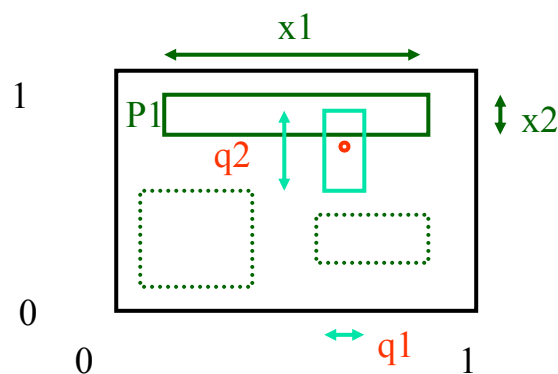
How many times will P1 be retrieved (unif. POINT queries)? A: $x1 \cdot x2$





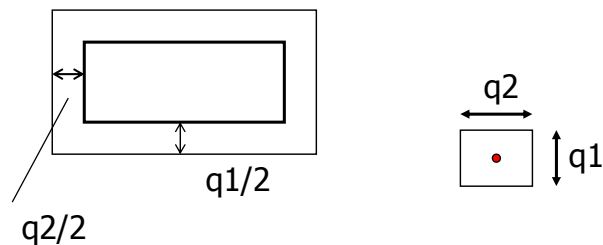
R-trees - performance analysis

- How many times will P1 be retrieved (unif. queries of size $q_1 \times q_2$)?



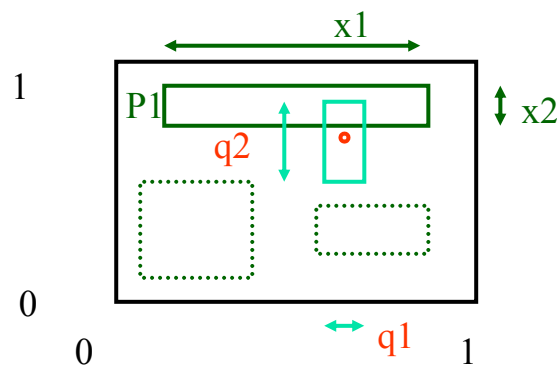
R-trees - performance analysis

- Minkowski sum



R-trees - performance analysis

- How many times will P1 be retrieved (unif. queries of size $q_1 \times q_2$)? A: $(x_1 + q_1)(x_2 + q_2)$



R-trees - performance analysis

- Thus, given a tree with n nodes ($i=1, \dots, n$) we expect

$$\begin{aligned}
 DA(q_1, q_2) &= \sum_i^n (x_{i,1} + q_1)(x_{i,2} + q_2) \\
 &= \sum_i^n x_{i,1} * x_{i,2} + \\
 &\quad q_1 \sum_i^n x_{i,2} + q_2 \sum_i^n x_{i,1} \\
 &\quad + q_1 * q_2 * n
 \end{aligned}$$



R-trees - performance analysis

- Thus, given a tree with n nodes ($i=1, \dots, n$) we expect

$$\begin{aligned}
 DA(q_1, q_2) &= \sum_i^n (x_{i,1} + q_1)(x_{i,2} + q_2) \\
 &= \sum_i^n x_{i,1} * x_{i,2} + \quad \longrightarrow \text{'volume'} \\
 &\quad q_1 \sum_i^n x_{i,2} + q_2 \sum_i^n x_{i,1} \longrightarrow \text{'surface area'} \\
 &\quad + q_1 * q_2 * n \quad \longrightarrow \text{count}
 \end{aligned}$$



R-trees - performance analysis

Observations:

- for point queries: only volume matters
- for horizontal-line queries: ($q_2=0$): vertical length matters
- for large queries ($q_1, q_2 \gg 0$): the count N matters
- overlap: does not seem to matter (but it is related to area)
- formula: easily extendible to n dimensions



R-trees - performance analysis

Conclusions:

- splits should try to minimize area and perimeter
- ie., we want few, small, square-like parent MBRs
- rule of thumb: shoot for queries with $q_1=q_2 = 0.1$ (or $=0.05$ or so).



More general Model

- What if we have only the dataset $\mathcal{D}$ and the set of queries $\mathcal{S}$?
- We should “predict” the structures of a “good” R-tree for this dataset. Then use the previous model to estimate the average query performance for $\mathcal{S}$
- For point dataset, we can use the Fractal Dimension to find the “average” structure of the tree
 - (More in the [FK94] paper)



Unifrom dataset

- Assume that the dataset (that contains only rectangles) is uniformly distributed in space.
- **Density** of a set of N MBRs is the average number of MBRs that contain a given point in space. OR the total area covered by the MBRs over the area of the work space.
- N boxes with average size $\mathbf{s} = (s_1, s_2)$, $D(N, \mathbf{s}) = N s_1 s_2$
- If $s_1 = s_2 = s$, then:

$$D = N s^2 \Rightarrow s = \sqrt{\frac{D}{N}}$$



Density of Leaf nodes

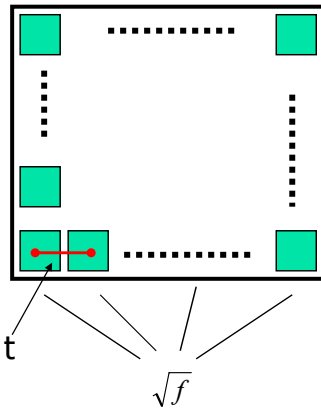
- Assume a dataset of N rectangles. If the average page capacity is f, then we have $N_{ln} = N/f$ leaf nodes.
- If D_1 is the density of the leaf MBRs, and the average area of each leaf MBR is s^2 , then:

$$D_1 = \frac{N}{f} s^2 \Rightarrow s_1 = \sqrt{D_1 \frac{f}{N}}$$

- So, we can estimate s_1 , from N, f, D_1
- We need to estimate D_1 from the dataset's density...



Estimating D_1



Consider a leaf node that contains f MBRs.

Then for each side of the leaf node MBR we have: $\sqrt{f}$ MBRs

Also, N_{in} leaf nodes contain N MBRs, uniformly distributed.

The average distance between the centers of two consecutive MBRs is $t = \frac{1}{\sqrt{N}}$ (assuming $[0,1]^2$ space)



Estimating D_1

- Combining the previous observations we can estimate the density at the leaf level, from the density of the dataset:

$$D_1 = \left\{1 + \frac{\sqrt{D_0} - 1}{\sqrt{f}}\right\}^2$$

- We can apply the same ideas recursively to the other levels of the tree.



R-trees–performance analysis

- Assuming Uniform distribution:

$$DA(q) = 1 + \sum_{j=1}^{1+h} \{(\sqrt{D_j} + q\sqrt{\frac{N}{f^j}})^2\}$$

where $D_j = \{1 + \frac{\sqrt{D_{j-1}} - 1}{\sqrt{f}}\}^2$ and $D_0 = D$

And D is the density of the dataset, f the fanout [TS96], N the number of objects



References

- Christos Faloutsos and Ibrahim Kamel. "Beyond Uniformity and Independence: Analysis of R-trees Using the Concept of Fractal Dimension". Proc. ACM PODS, 1994.
- Yannis Theodoridis and Timos Sellis. "A Model for the Prediction of R-tree Performance". Proc. ACM PODS, 1996.